

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра информационной безопасности

ОТЧЁТ
по лабораторной работе №5
по дисциплине: «Компьютерная графика»
Тема: «Исследование алгоритмов выявления видимости сложных сцен»

Студент гр. 4364	_____	Ефремов Э. А.
Студент гр. 4364	_____	Поляков Н. Н.
Преподаватель	_____	Матвеева И. В.

Санкт-Петербург

2026

Вариант № 3 (тема 5). Обеспечить реализацию видимости совокупности произвольных многогранников на основе алгоритма сортировки граней по z-координате.

ЗАДАНИЕ

Разработать программный продукт на языке высокого уровня, реализующий выявление видимости совокупности произвольных многогранников методом сортировки граней по глубине (алгоритм художника) и их визуализацию на плоскости экрана. Предусмотреть:

- формирование трёхмерной сцены из нескольких многогранников (кубов и пирамид) с добавлением и удалением тел пользователем;
- вычисление для каждой грани средней z-координаты, а также минимума и максимума z и вывод этих величин в таблицу;
- сортировку всех граней сцены по средней z-координате и отрисовку в порядке «дальние грани → ближние грани», при которой ближняя грань перекрывает ранее нарисованные;
- выявление обращённых к наблюдателю (лицевых) и обратных граней, отсечение обратных граней по выбору пользователя;
- поворот сцены вокруг осей X и Y, изменение масштаба, отображение каркаса и подписей граней;
- сохранение и загрузку произвольной сцены во внешний файл.

Специализированные графические библиотеки типа OpenGL не использовать.

ЦЕЛЬ РАБОТЫ

Изучить методы выявления видимости объектов сложной сцены; освоить алгоритм сортировки поверхностей по глубине (алгоритм художника) и определение лицевых граней по ориентации нормали; исследовать точность и границы применимости этих методов путём сравнения с эталонным пофрагментным разрешением видимости (z-буфером); реализовать соответствующие алгоритмы и интерактивный

интерфейс пользователя для управления составом сцены и параметрами наблюдения.

МАТЕМАТИЧЕСКАЯ ОСНОВА

1. Модель сцены и ориентация граней

Сцена представляет собой совокупность многогранников. Каждый многогранник описывается списком вершин $P(i) = (x, y, z)$ и списком граней; грань задаётся упорядоченным набором индексов вершин и названием. Вершины грани перечисляются против часовой стрелки при взгляде с внешней стороны тела, поэтому внешняя нормаль грани вычисляется через векторное произведение:

$$\mathbf{n} = (P_1 - P_0) \times (P_2 - P_0) \quad (1)$$

где $P(0)$, $P(1)$, $P(2)$ – первые три вершины грани. Направление нормали однозначно определяет ориентацию грани в пространстве, что используется далее для отсекающих обратных граней. Скалярное произведение векторов, необходимое для этой проверки, вычисляется стандартно:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z \quad (2)$$

В работе используются два типа тел: куб (8 вершин, 6 четырёхугольных граней) и пирамида (4 вершины основания и вершина, 5 граней). Сцена примера содержит три многогранника – всего 17 граней, что позволяет наглядно проверить работу сортировки.

2. Система координат наблюдателя

Наблюдение сцены выполняется в два этапа. Сначала пользователь поворачивает сцену вокруг осей X и Y ; матрицы поворота имеют вид

$$R_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix} \quad (3)$$

$$R_y(\beta) = \begin{pmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{pmatrix} \quad (4)$$

Затем к повернутой сцене применяется фиксированный наклон камеры (изометрический разворот), задаваемый углами $\psi = -0,55$ рад ($-31,5^\circ$) и $\theta = 0,35$ рад ($20,1^\circ$):

$$\begin{aligned}x_1 &= x \cos \psi + z \sin \psi, & z_1 &= -x \sin \psi + z \cos \psi, \\y_2 &= y \cos \theta - z_1 \sin \theta, & z_2 &= y \sin \theta + z_1 \cos \theta\end{aligned}\quad (5)$$

Полученные координаты $(x(1), y(2), z(2))$ – это координаты точки в системе наблюдателя. Ось Z этой системы направлена от наблюдателя вглубь сцены, поэтому чем больше z , тем ближе точка к наблюдателю. Глубина точки выражается через исходные координаты линейной функцией:

$$d(x, y, z) = -\cos \theta \sin \psi x + \sin \theta y + \cos \theta \cos \psi z \quad (6)$$

Численно для принятых углов наклона камеры:

$$d \approx 0,491 x + 0,343 y + 0,801 z \quad (7)$$

Именно эта величина определяет, какая из двух точек сцены находится ближе к наблюдателю. Существенно, что глубина зависит не только от z , но и от x и y : вклад координаты x достигает 49 % веса. Этот факт важен для анализа точности алгоритма сортировки (см. раздел 6).

Ортогональное проецирование точки системы наблюдателя на холст шириной W и высотой H выполняется по формулам

$$x_{\text{px}} = \frac{W}{2} + k x, \quad y_{\text{px}} = \frac{H}{2} - k y \quad (8)$$

где k – коэффициент масштаба; знак «минус» во второй формуле учитывает направление оси Y экрана вниз.

3. Выявление видимости грани по ориентации нормали

Грань обращена к наблюдателю (является лицевой), если её внешняя нормаль направлена в сторону наблюдателя, то есть имеет положительную z -компоненту в системе наблюдателя:

$$n_z^{\text{cam}} > 0 \quad (9)$$

Не менее важна и обратная задача: если нормаль направлена от наблюдателя ($n(z) < 0$), грань является обратной и при включённом

отсечении обратных граней не рисуется. Для выпуклого тела отсечение удаляет примерно половину граней:

$$N_{\text{cull}} \approx \frac{1}{2} N_{\text{total}} \quad (10)$$

что вдвое сокращает объём отрисовки. При этом необходимо подчеркнуть, что проверку следует выполнять именно в системе наблюдателя: использование z-компоненты нормали в системе координат объекта (до наклона камеры) даёт ошибочный результат, что подтверждено экспериментально (раздел 5.4).

4. Сортировка граней по глубине (алгоритм художника)

Алгоритм художника основан на упорядоченной отрисовке поверхностей: сначала изображаются наиболее удалённые грани, затем более близкие; каждая следующая грань перекрывает ранее нарисованные. Для каждой грани вычисляются средняя глубина, а также минимальная и максимальная глубины её вершин:

$$z_{\text{avg}} = \frac{1}{m} \sum_{i=1}^m z_i, \quad z_{\min} \leq z_{\text{avg}} \leq z_{\max} \quad (11)$$

Все грани сцены сортируются по возрастанию средней глубины, что соответствует порядку «дальние → ближние»:

$$z_{\text{avg}}^{(1)} \leq z_{\text{avg}}^{(2)} \leq \dots \leq z_{\text{avg}}^{(N)} \quad (12)$$

Затем грани последовательно закрашиваются на холсте в полученном порядке. Величины $z(\min)$ и $z(\max)$ позволяют оценить протяжённость грани по глубине и используются для контроля: если диапазоны глубин двух граней перекрываются, сортировка по средней глубине может дать неверный порядок перекрытия. Таблица, выводимая в интерфейсе программы, содержит номер грани в порядке обработки, её название и значения $z(\text{cp})$, $z(\min)$, $z(\max)$.

Порядок «дальние → ближние» полностью совпадает с описанным в методических указаниях принципом: поверхности сортируются в порядке

уменьшения глубины и преобразуются в стандарт развёртки, начиная с наиболее глубокой.

5. Ограничения метода сортировки целых граней

Сортировка по средней глубине разрешает перекрытие только на уровне целых граней. Если два тела пересекаются или их диапазоны глубин взаимно перекрываются, единый порядок граней может не существовать: для одних участков изображения ближе оказывается одна грань, для других – другая. Методические указания отмечают этот случай особо: при обнаружении перекрытия по глубине необходимы дополнительные сравнения и возможно переупорядочивание поверхностей.

Предельным случаем является взаимное проникновение тел: часть грани одного тела должна быть видна, а часть – закрыта другим телом. Никакая сортировка целых граней не даёт правильного результата в этом случае; задача решается методами пространства изображений (z-буфер, построчное сканирование, деление окна). Для количественной оценки погрешности в работе использован эталонный метод – z-буфер, в котором для каждого пикселя хранится наибольшая достигнутая глубина. Глубина точки плоской грани вычисляется по уравнению плоскости:

$$z(x, y) = ax + by + c \quad (13)$$

где коэффициенты a , b , c определяются по трём вершинам грани. Сравнение изображения, полученного алгоритмом художника, с эталонным изображением z-буфера позволяет измерить долю ошибочно закрашенных пикселей (раздел 5.5).

ХОД РАБОТЫ

1. Средства реализации

Программа реализована на языке Python. Использована только стандартная библиотека языка: `math` (тригонометрические функции), `tkinter` и `ttk` (графический интерфейс, холст, таблица сортировки), `dataclasses`

(описание вершин, граней и многогранников), json (обмен сценой с внешним файлом), typing. Сторонние библиотеки не применяются: файл requirements.txt не содержит зависимостей. Специализированные графические библиотеки типа OpenGL не использовались – закраска граней выполняется примитивом `create_polygon` холста Tk.

Работа выполнена в два этапа. На первом этапе исследовалась исходная версия программы (каталог `kg5`), для которой проведены модульные тесты, численный анализ видимости и сортировки, а также сравнение с эталонным z-буфером. В ходе исследования выявлены четыре дефекта, влияющие на выполнение задания (инвертированный признак видимости, сортировка по z-координате объекта вместо глубины наблюдателя, неработающий флажок подписей и выход элементов интерфейса за пределы окна), и два оформленных дефекта вёрстки. На втором этапе создана исправленная версия (каталог `kg5_fixed`), в которой дефекты устранены; все приведённые ниже иллюстрации работы программы относятся к исправленной версии, а исходная версия сохранена для сравнения.

Особенности запуска и средств, обеспечивающих работу программы. Локальный запуск выполняется командой `python3 app/main.py` и требует интерпретатора Python 3.10+ со встроенным модулем `tkinter`. Модульные тесты запускаются интерпретатором `pytest` (`python3 -m pytest -q tests`) либо прямым вызовом тестовых функций. Для демонстрации работы на машине без графической подсистемы подготовлен контейнер: в образе разворачивается виртуальный дисплей `Xvfb` (1280×800×24), сервер `x11vnc` и веб-клиент `noVNC`, интерфейс открывается в браузере по адресу `http://localhost:6080/vnc.html`. Сборка и запуск выполняются командами `docker compose up --build`.

2. Структура программы и ключевые функции

Программа состоит из модели сцены, вычислительного ядра и класса интерфейса. Состав функций и методов приведён в таблице 1.

Таблица 1. Функции и методы программы

Имя	Назначение
Point3D	точка пространства; метод rotate(rx, ry) – поворот вокруг осей X и Y по формулам (3), (4)
Face, Polyhedron	грань (индексы вершин, название) и многогранник (вершины, грани, цвет заливки)
dot, cross, subtract	векторные операции: скалярное (2) и векторное (1) произведения, разность векторов
face_normal(vertices)	внешняя нормаль грани по формуле (1)
create_box, create_pyramid, create_scene	построение куба, пирамиды и сцены примера
to_camera(p)	перевод точки в систему наблюдателя по формулам (5)
project_camera(p, W, H, k)	ортогональная проекция точки системы наблюдателя (8)
project(p, W, H, k)	проекция точки мировых координат (наклон камеры и проекция)
build_face_records(scene, rx, ry)	ядро алгоритма: для каждой грани вычисляются глубина, нормаль, признак видимости; список сортируется по z(cp)
App._refresh	отрисовка сцены: грани в порядке сортировки, каркас, подписи и обновление таблицы
App.fit_scale	подбор масштаба, при котором сцена целиком помещается на холсте
App.add_cube, add_pyramid, remove_selected	изменение состава сцены
App.save_scene, load_scene	обмен сценой с файлом JSON

Ниже приведены листинги ключевых фрагментов вычислительного ядра.

Вычисление нормали грани (формула 1) и скалярного произведения (2):

```
def dot(a: Point3D, b: Point3D) -> float:
    return a.x * b.x + a.y * b.y + a.z * b.z

def cross(a: Point3D, b: Point3D) -> Point3D:
    return Point3D(
        a.y * b.z - a.z * b.y,
        a.z * b.x - a.x * b.z,
        a.x * b.y - a.y * b.x,
    )

def subtract(a: Point3D, b: Point3D) -> Point3D:
    return Point3D(a.x - b.x, a.y - b.y, a.z - b.z)

def face_normal(vertices: List[Point3D]) -> Point3D:
    if len(vertices) < 3:
        return Point3D(0, 0, 0)
    return cross(subtract(vertices[1], vertices[0]),
        subtract(vertices[2], vertices[0]))
```

Перевод точки в систему координат наблюдателя (формулы 5):

```
# Фиксированная ориентация наблюдения (наклон камеры). Одна и та же для
# сортировки граней, определения видимости и проецирования.
VIEW_YAW = -0.55
VIEW_PITCH = 0.35
```

```
def to_camera(p: Point3D, yaw: float = VIEW_YAW, pitch: float = VIEW_PITCH) -> Point3D:
    """Перевод точки в систему координат наблюдателя (после поворота сцены).
```

В этой системе ось Z направлена от наблюдателя вглубь сцены, поэтому:

```
    * большая z - точка ближе к наблюдателю;
    * грань видима, если внешняя нормаль имеет положительную z-компоненту.
```

```
    """
    cy, sy = math.cos(yaw), math.sin(yaw)
    x1 = p.x * cy + p.z * sy
    z1 = -p.x * sy + p.z * cy
    cx, sx = math.cos(pitch), math.sin(pitch)
    y2 = p.y * cx - z1 * sx
    z2 = p.y * sx + z1 * cx
    return Point3D(x1, y2, z2)
```

```
def project_camera(p: Point3D, width: int, height: int, scale: float):
    """Ортогональная проекция точки, уже переведённой в систему наблюдателя."""
    return width / 2 + p.x * scale, height / 2 - p.y * scale
```

```
def project(p: Point3D, width: int, height: int, scale: float, yaw: float = VIEW_YAW, pitch: float = VIEW_PITCH):
    """Проекция точки мировых координат: наклон камеры, затем ортогональная проекция."""
    q = to_camera(p, yaw, pitch)
    x, y = project_camera(q, width, height, scale)
```

Ядро алгоритма – вычисление глубины, нормали и признака видимости каждой грани с последующей сортировкой по средней глубине (формулы 9, 11, 12):

```
def build_face_records(scene: List[Polyhedron], rx: float, ry: float) -> List[FaceRecord]:
    records: List[FaceRecord] = []
    for poly in scene:
        # Поворот сцены пользователем и перевод в систему наблюдателя.
```

```

        transformed = [to_camera(p.rotate(rx, ry)) for p in
poly.vertices]
        for face in poly.faces:
            pts = [transformed[i] for i in face.indices]
            nz = face_normal(pts).z
            # Наблюдатель расположен на +Z системы наблюдателя,
поэтому грань
            # видима, если её внешняя нормаль направлена к
наблюдателю (nz > 0).
            records.append(
                FaceRecord(
                    poly.name,
                    face.name,
                    face.indices,
                    pts,
                    sum(p.z for p in pts) / len(pts),
                    min(p.z for p in pts),
                    max(p.z for p in pts),
                    nz,
                    nz > 0,
                )
            )
        # Painter algorithm: farthest average z first, nearest last.
        # This is the requested depth sorting by z-coordinate.
        records.sort(key=lambda r: r.z_avg)
        return records

```

Отрисовка граней в полученном порядке «дальние → ближние»:

```

def _refresh(self):
    if not hasattr(self, "canvas"):
        return
    self.canvas.delete("all")
    w = max(self.canvas.winfo_width(), 500)
    h = max(self.canvas.winfo_height(), 400)
    self._draw_axes(w, h)
    records = build_face_records(self.scene, self.rx,
self.ry)

    # Painter algorithm: the records are already sorted from
far to near.
    for rec in records:
        if self.use_backface.get() and not rec.visible:
            continue
        poly = next(p for p in self.scene if p.name ==
rec.poly_name)
        points = []
        for p in rec.vertices:
            # уже в
системе наблюдателя
            q = project_camera(p, w, h, self.scale.get())
            points.extend([q[0], q[1]])
        is_selected = False
        color = poly.fill
        if not rec.visible:

```

```

        color = "#263244"
        outline = "#e5e7eb" if rec.visible else "#475569"
        width = 1
        if self.show_hidden.get() or rec.visible:
            self.canvas.create_polygon(points, fill=color,
            outline=outline, width=width)
            if self.show_labels.get():
                cx = sum(points[0::2]) / len(rec.vertices)
                cy = sum(points[1::2]) / len(rec.vertices)
                self.canvas.create_text(cx, cy,
            text=f"{rec.poly_name}: {rec.face_name}", fill="#ffffff",
            font=("TkDefaultFont", 8))

```

3. Интерфейс пользователя

Главное окно программы (рисунок 1) состоит из трёх областей: слева – панель управления, в центре – область визуализации, справа – таблица результата сортировки граней; внизу расположена статусная строка с текущими параметрами сцены и наблюдения.

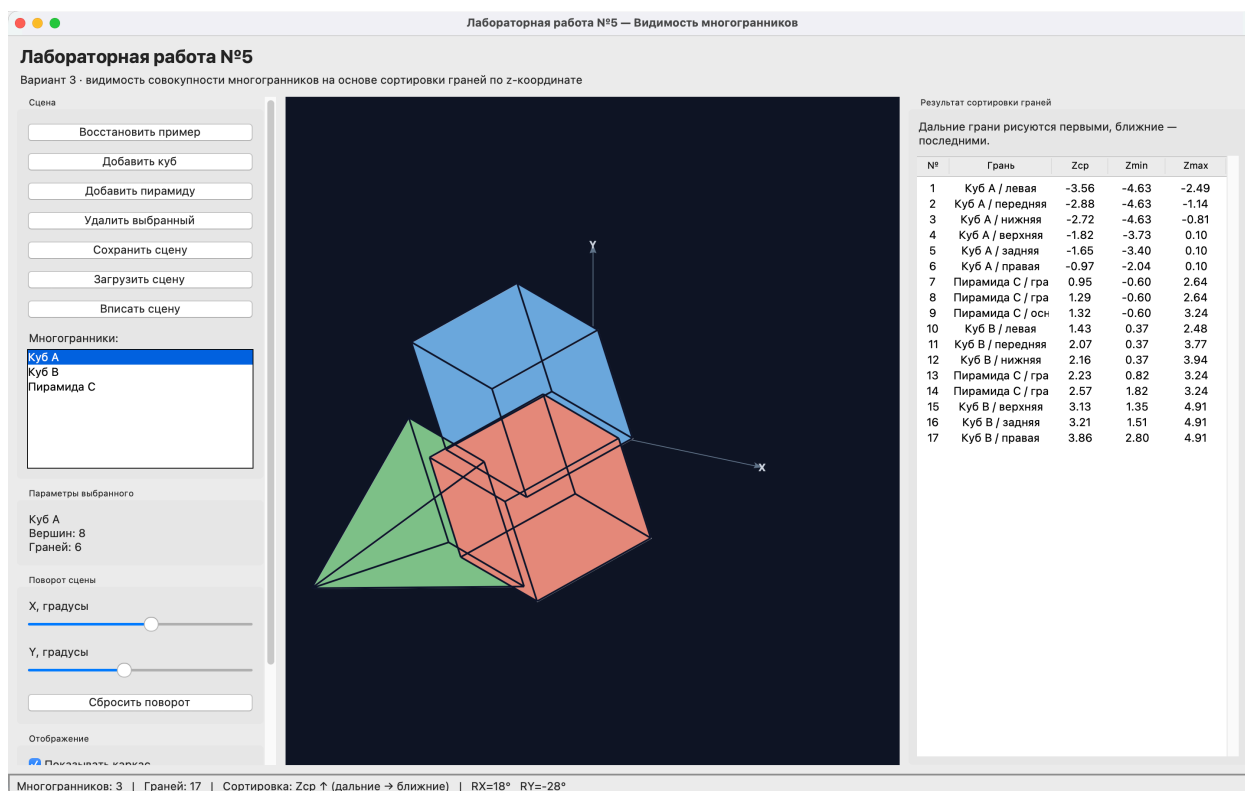


Рисунок 1. Главное окно программы: сцена примера, сортировка 17 граней

Назначение элементов управления приведено в таблице 2.

Таблица 2. Элементы управления программой

Группа	Элемент	Назначение
Сцена	кнопки «Восстановить пример», «Добавить куб», «Добавить пирамиду», «Удалить	изменение состава сцены, обмен сценой с файлом JSON, автоматический подбор масштаба

Группа	Элемент	Назначение
	выбранный», «Сохранить сцену», «Загрузить сцену», «Вписать сцену»; список многогранников	
Параметры выбранного	текстовая справка	имя многогранника, число вершин и граней
Поворот сцены	ползунки «Х, градусы» и «У, градусы» ($-180...+180^\circ$), кнопка «Сбросить поворот»	поворот сцены вокруг осей Х и У с немедленной перерисовкой
Отображение	флажки «Показывать каркас», «Показывать скрытые грани», «Подписывать грани», «Отсечение обратных граней»; ползунок «Масштаб»	управление составом изображения и отсечением обратных граней
Таблица результата сортировки	столбцы «№», «Грань», «Zср», «Zmin», «Zmax»	порядок обработки граней и их глубины; дальние грани находятся в начале списка

Область визуализации отображает закрашенные грани многогранников, каркас (тонкие тёмные линии) и, при необходимости, подписи граней. Цвет заливки тела задаётся при его создании, обратные грани при включённом режиме показа скрытых граней изображаются тёмно-синим цветом, что позволяет визуально отличать лицевые и обратные грани.

Обмен сценой выполняется в формате JSON: для каждого многогранника сохраняются имя, цвет, список вершин и список граней с индексами и названиями. Пример файла приведён в приложении Б. При загрузке проверяется корректность структуры; при ошибке выводится диалоговое окно с описанием причины.

4. Алгоритм работы программы

- 1) При запуске создаётся окно, панель управления, холст и таблица сортировки; формируется сцена примера из трёх многогранников; устанавливаются углы поворота $\alpha = 18^\circ$, $\beta = -28^\circ$ и выполняется первая отрисовка.
- 2) После первого показа холста автоматически подбирается масштаб, при котором сцена целиком помещается в область визуализации.
- 3) Любое действие пользователя (поворот сцены, изменение состава сцены, переключение флажков, изменение масштаба) вызывает метод `redraw/_refresh`, полностью перестраивающий изображение и таблицу.

- 4) Для каждой грани сцены вычисляются вершины в системе наблюдателя, внешняя нормаль, признак видимости, средняя, минимальная и максимальная глубины.
- 5) Список граней сортируется по средней глубине по возрастанию – от наиболее удалённых к наиболее близким.
- 6) Грани последовательно закрашиваются на холсте: если включён режим отсечения обратных граней, грани с отрицательной z-компонентой нормали пропускаются; если отключён режим показа скрытых граней, обратные грани также не рисуются.
- 7) Поверх закрашенных граней при необходимости выводятся каркас (все рёбра многогранников) и подписи граней.
- 8) Таблица сортировки заполняется в порядке обработки граней, статусная строка обновляет сведения о числе многогранников, граней, угле поворота и режиме сортировки.

Формально алгоритм можно записать следующим псевдокодом:

```

ПОСТРОИТЬ_СЦЕНУ(scene, alpha, beta):
    records = []
    для каждого многогранника poly из scene:
        для каждой грани face из poly.faces:
            pts = [to_camera(P.rotate(alpha, beta)) для P из
face]
            n = (pts[1]-pts[0]) x (pts[2]-pts[0]) #
формула (1)
            z_avg = среднее(pts.z); z_min = min(pts.z); z_max =
max(pts.z)
            visible = (n.z > 0) #
формула (9)
            records.добавить(pts, z_avg, z_min, z_max, visible)
            records.сортировать(по z_avg возрастанию) #
формула (12)

    для каждой записи rec из records: #
порядок художника
        если rec.visible или показывать_скрытые:
            если отсечение_обратных и не rec.visible: пропустить
            нарисовать_полигон(проекция(rec.pts), цвет)
            заполнить_таблицу(records)

```

5. Тестирование

Визуальные проверки. На рисунках 2–7 приведены снимки экрана программы для различных режимов отображения и ракурсов.

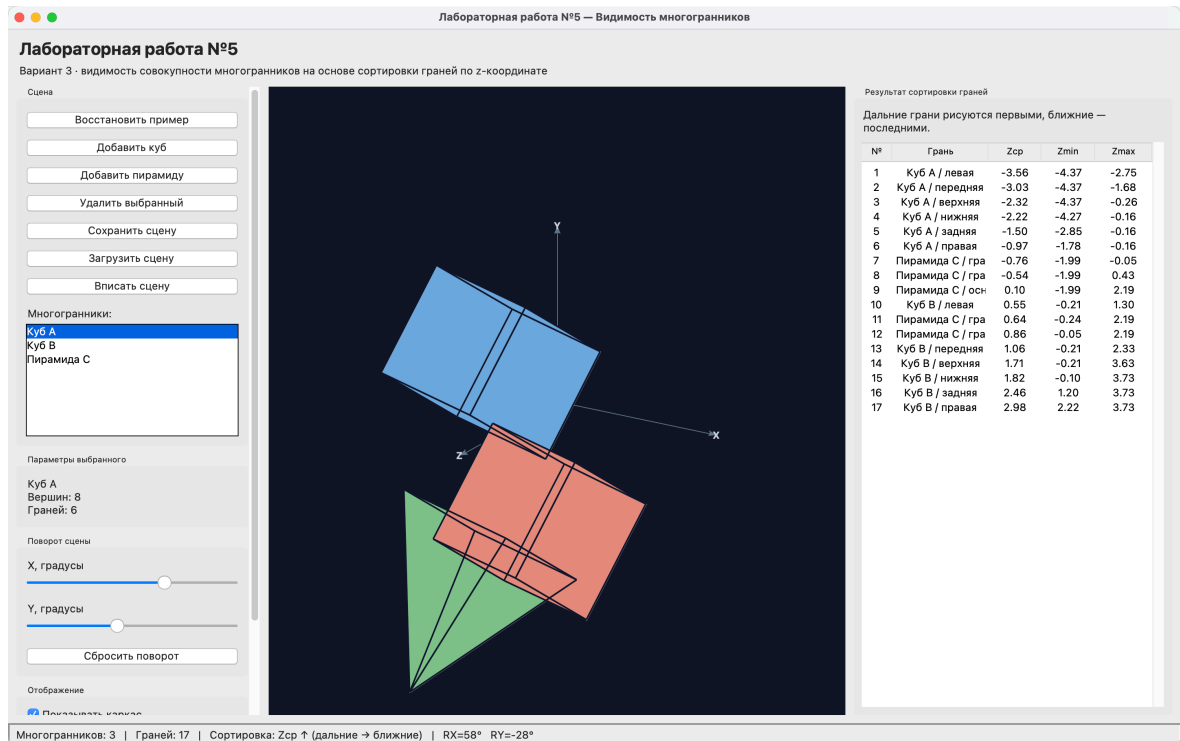


Рисунок 2. Поворот сцены вокруг оси X на 58°

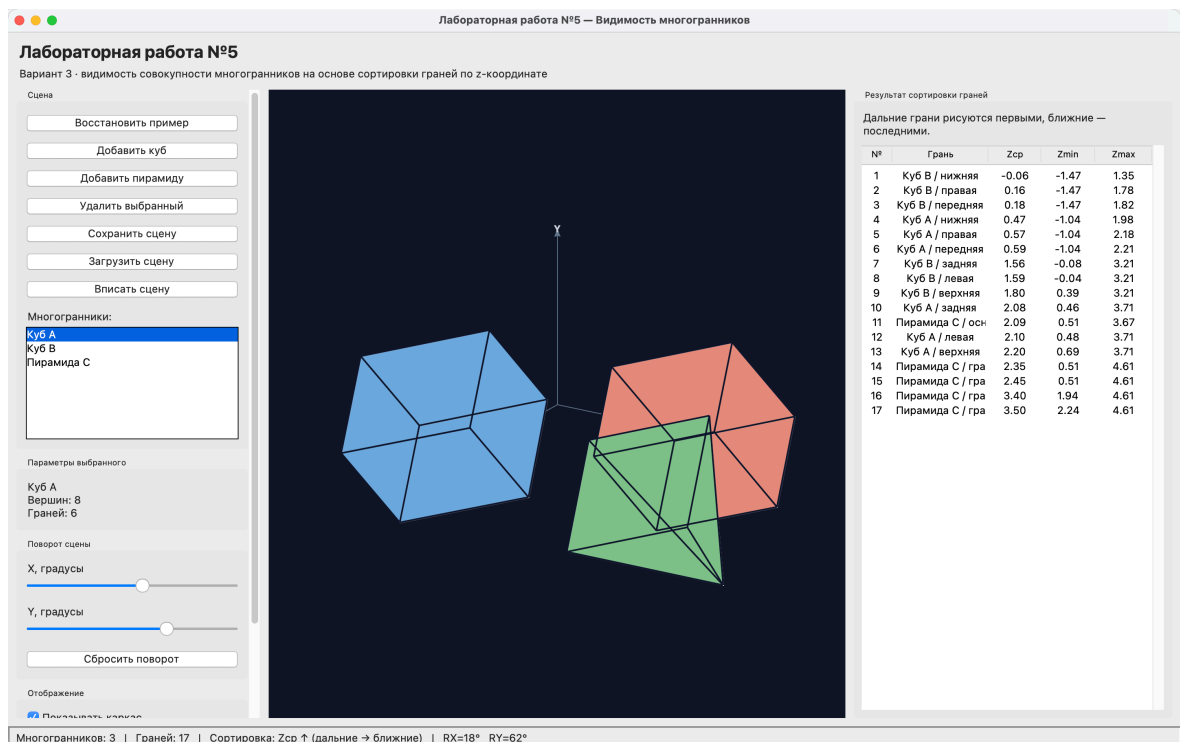


Рисунок 3. Поворот сцены вокруг оси Y на 62°

При повороте сцены (рисунки 2 и 3) таблица сортировки перестраивается: изменяются значения $z(cp)$, $z(min)$, $z(max)$ и порядок обработки граней, что соответствует изменению взаимного расположения тел относительно наблюдателя.

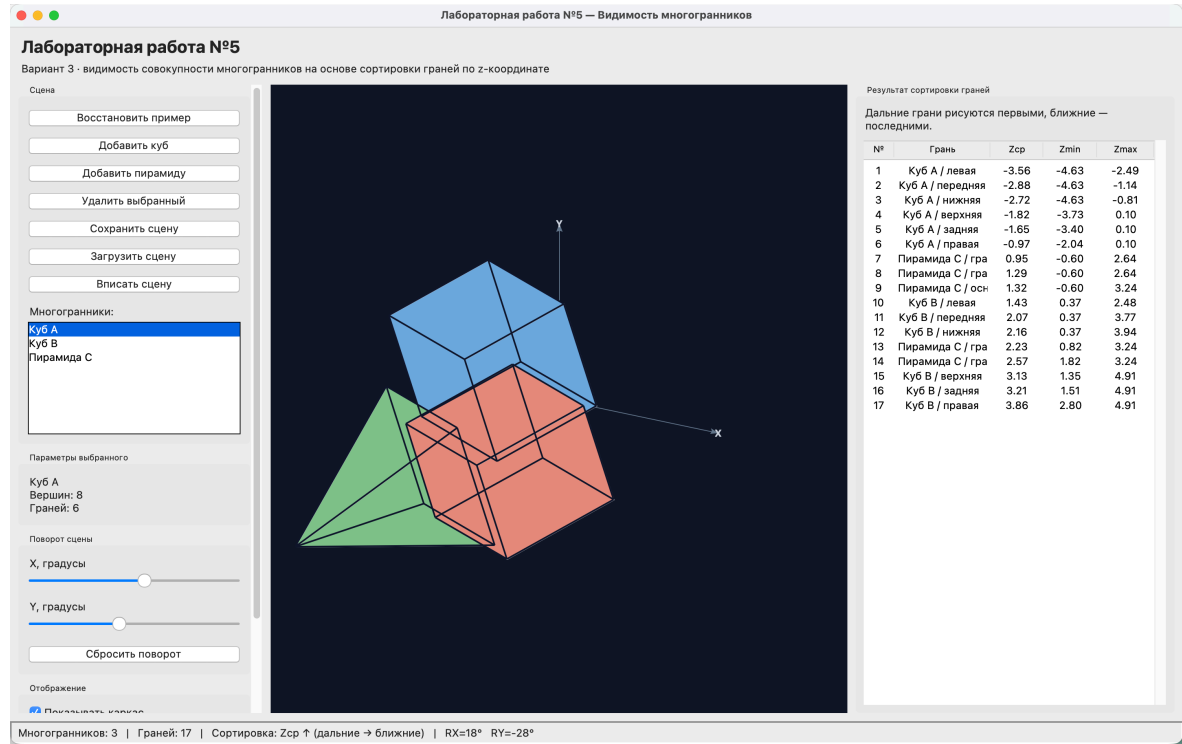


Рисунок 4. Режим показа скрытых граней: обратные грани закрашены тёмным цветом

На рисунке 4 включён режим показа скрытых граней: все 17 граней закрашиваются, при этом обратные грани изображаются тёмно-синим цветом и не закрывают лицевые грани благодаря правильному порядку отрисовки.

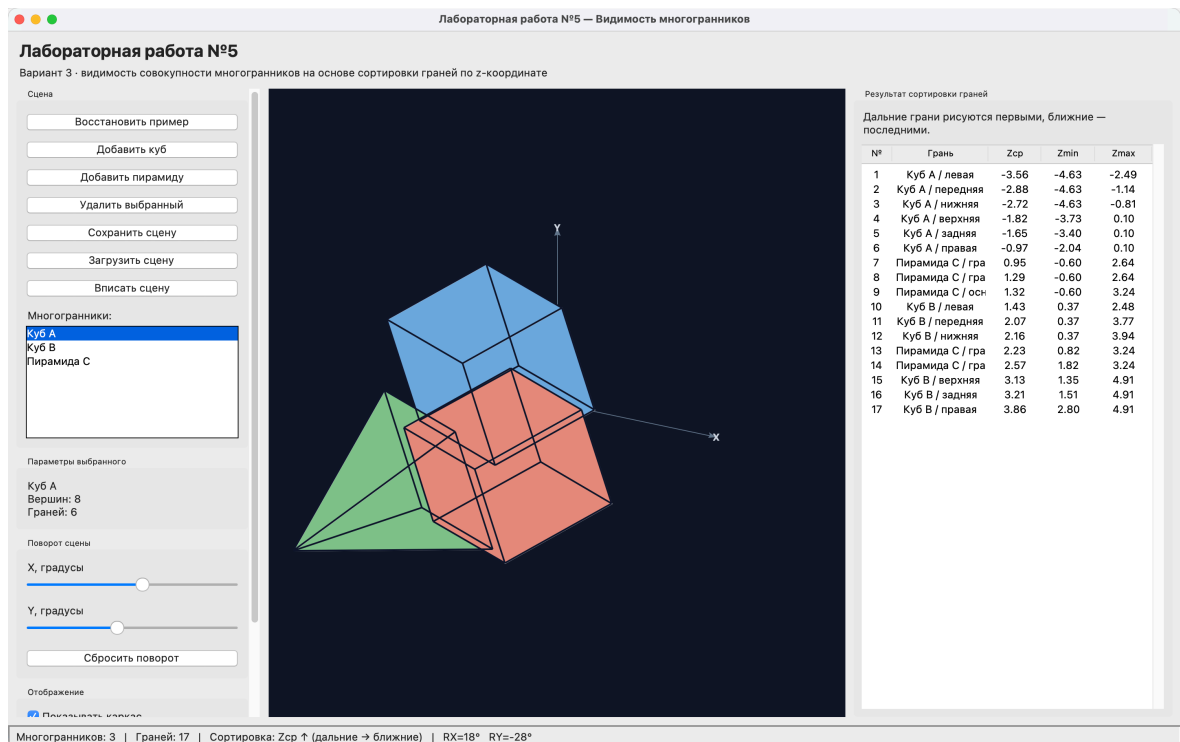


Рисунок 5. Отсечение обратных граней: изображены только лицевые грани

На рисунке 5 включено отсечение обратных граней: обратные грани не рисуются вовсе, что примерно вдвое сокращает число изображаемых граней (формула 10).

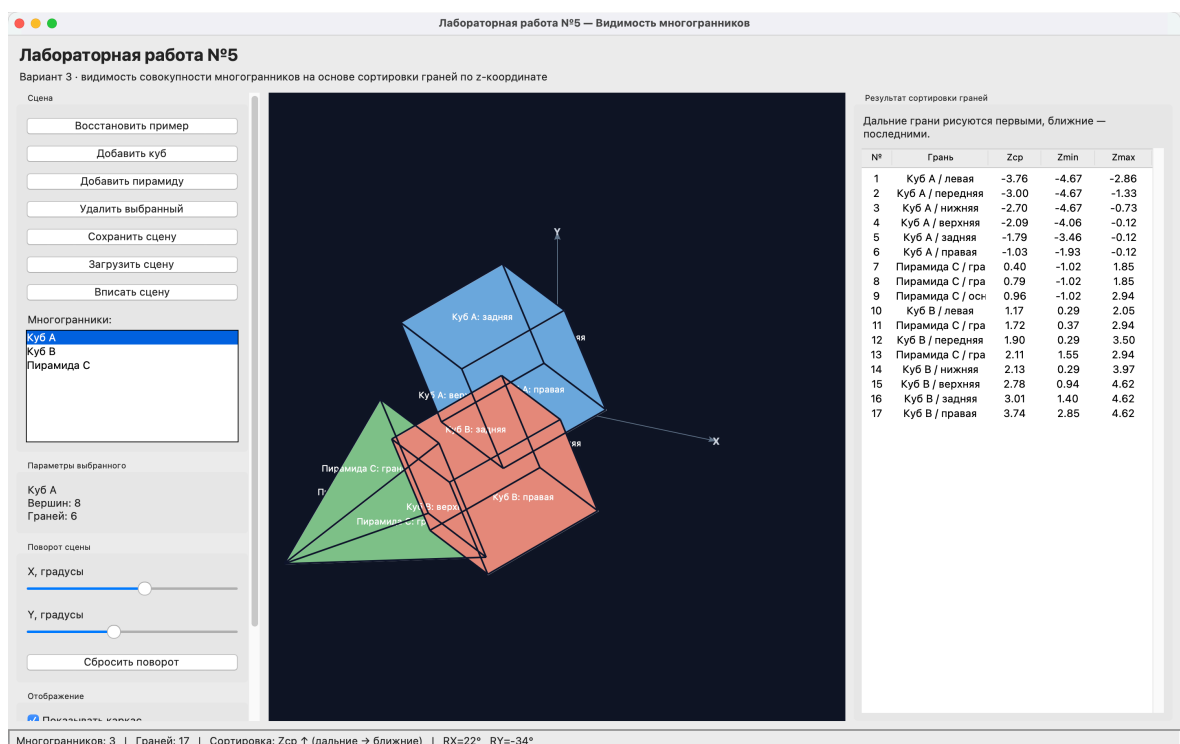


Рисунок 6. Режим подписей граней

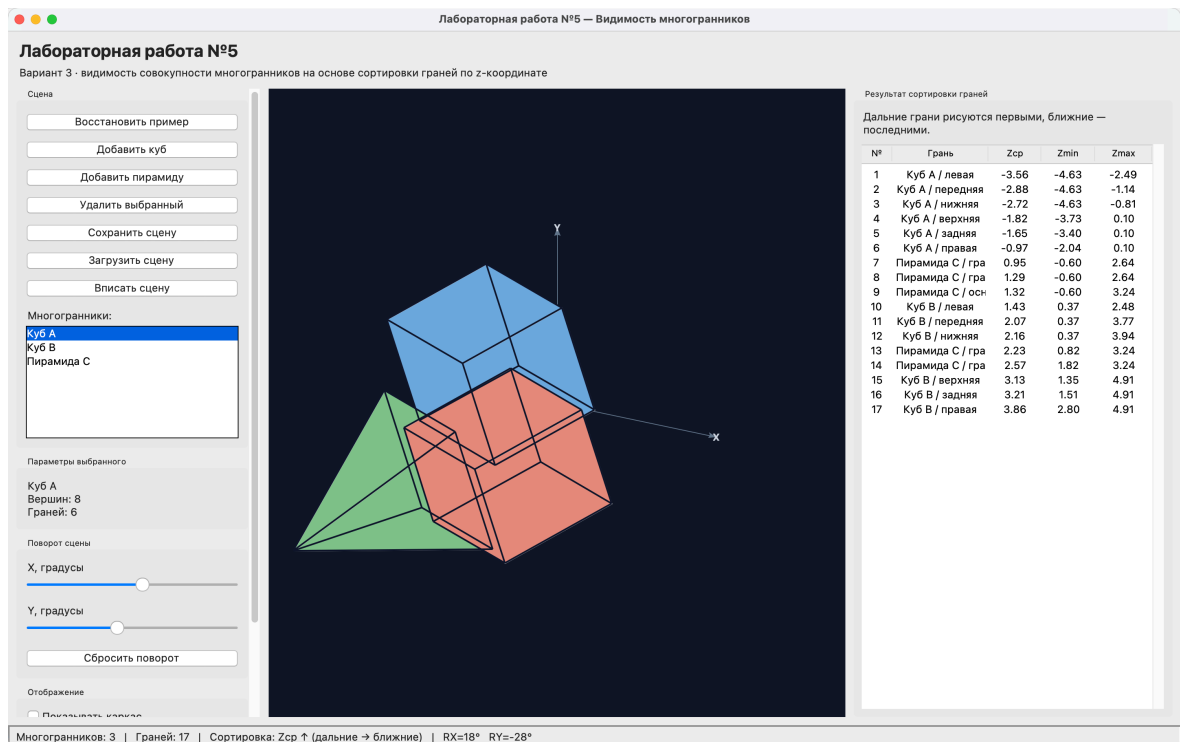


Рисунок 7. Изображение без каркаса (только закрашенные грани)

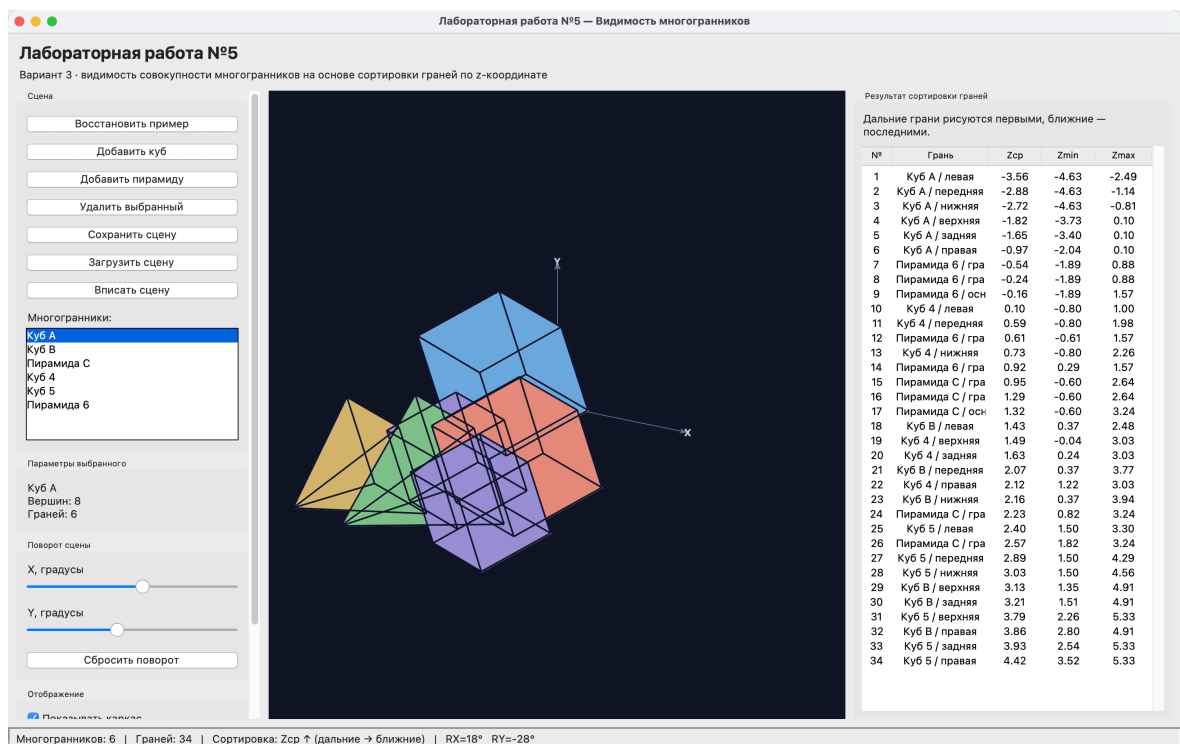


Рисунок 8. Сцена из шести многогранников: сортировка 34 граней

Программа работоспособна при произвольном числе тел: на рисунке 8 показана сцена из шести многогранников, для которой таблица содержит 34 строки, упорядоченные по возрастанию средней глубины.

5.5. Исследование границ применимости алгоритма. Алгоритм художника разрешает перекрытие целых граней, поэтому его погрешность зависит от взаимного расположения тел. Для количественной оценки изображение, построенное программой, сравнивалось с эталонным изображением, полученным методом z-буфера с пофрагментным разрешением видимости (формула 13). Результаты приведены в таблице 8.

Таблица 3. Расхождение алгоритма художника с эталонным z-буфером

Конфигурация сцены	Расхождение, пикселей (%), 900×700
Два куба, разнесённых по координате x (диапазоны глубин не перекрываются), 49 ракурсов	0 (0,000 %) – полное совпадение
Сцена примера (диапазоны глубин тел частично перекрываются), 7 ракурсов	от 2254 до 8905 (от 0,36 % до 1,41 %)
Два пересекающихся куба, 169 ракурсов	до 28 829 (4,58 %) – максимальное расхождение

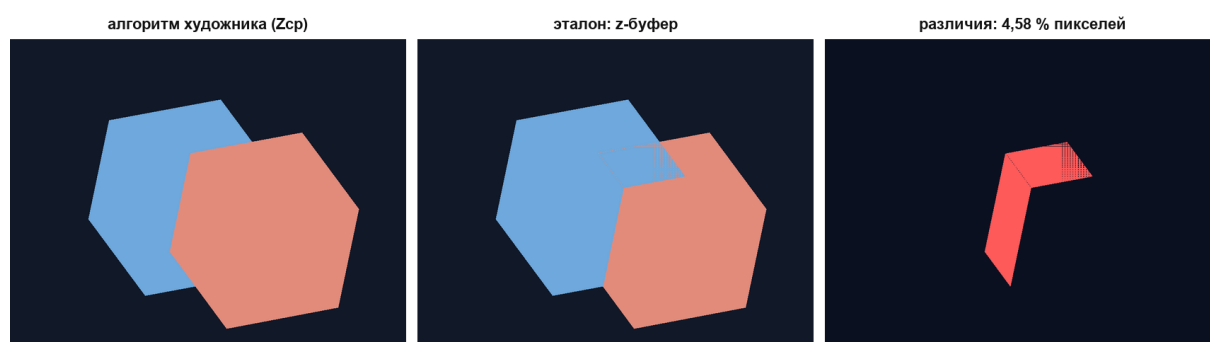


Рисунок 10. Пересекающиеся кубы: результат алгоритма художника, эталонный z-буфер и карта различий

Результаты показывают, что для тел, не перекрывающихся по глубине, алгоритм художника даёт результат, полностью совпадающий с эталонным (0 пикселей расхождения). Погрешность появляется в двух случаях: когда диапазоны глубин тел взаимно перекрываются (сцена примера, до 1,41 %) и когда тела взаимно проникают друг в друга (до 4,58 %). Второй случай принципиально неразрешим сортировкой целых граней: на рисунке 10 красным показаны пиксели, где порядок граней, выбранный алгоритмом художника, противоречит результату пофрагментного разрешения видимости. Для корректной визуализации пересекающихся тел необходимы методы пространства изображений (z-буфер, построчное сканирование,

деление окна), что соответствует указаниям методических материалов о необходимости дополнительных сравнений при перекрытии по глубине.

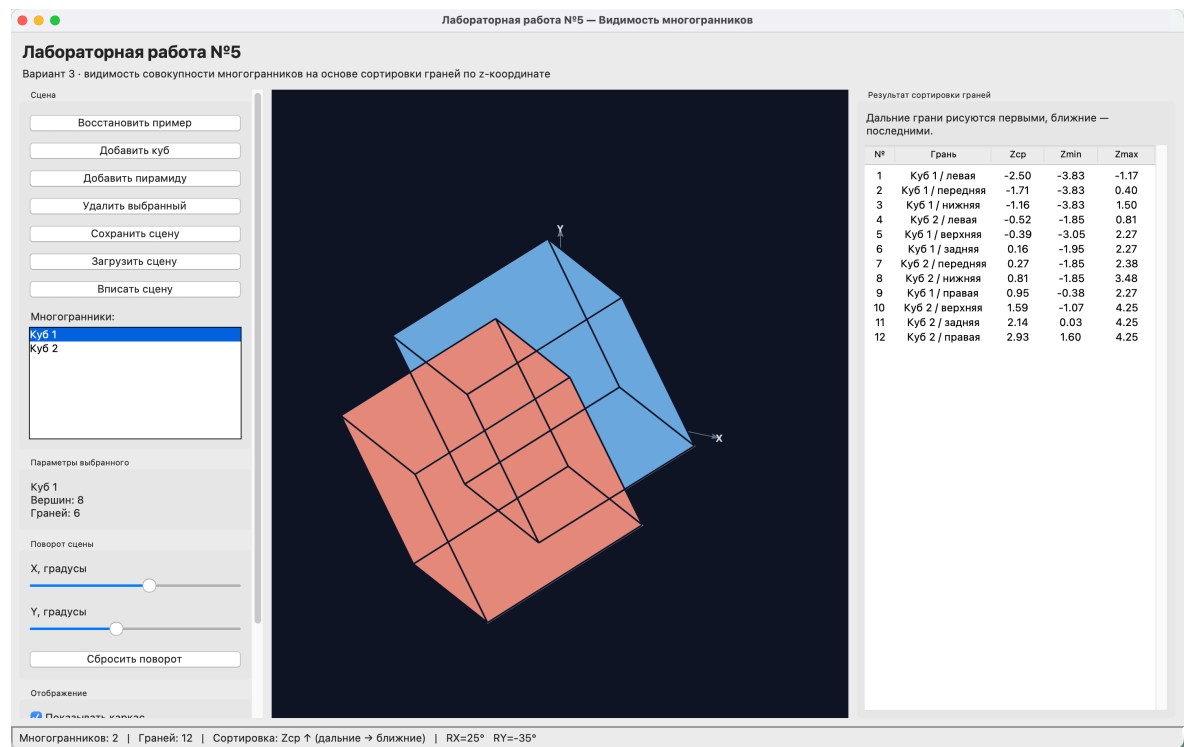


Рисунок 11. Пересекающиеся кубы в окне программы (режим показа скрытых граней)

ВЫВОД

В ходе выполнения лабораторной работы изучены методы выявления видимости объектов сложной сцены. Освоен алгоритм сортировки поверхностей по глубине (алгоритм художника): для каждой грани вычисляются средняя, минимальная и максимальная глубины, список граней сортируется в порядке «дальние $\rightarrow$ ближние» и отрисовывается последовательно, так что ближняя грань перекрывает ранее нарисованные. Изучено определение лицевых граней по ориентации внешней нормали в системе координат наблюдателя и отсечение обратных граней, сокращающее число изображаемых граней примерно вдвое.

Разработан программный продукт на языке Python, обеспечивающий: формирование произвольной сцены из кубов и пирамид, добавление и удаление тел; сортировку всех граней по средней глубине с выводом таблицы порядка обработки и значений $z(cp)$, $z(min)$, $z(max)$; поворот сцены вокруг осей X и Y; режимы показа скрытых граней и отсечения обратных граней; отображение каркаса и подписей граней; сохранение и загрузку сцены в формате JSON; автоматический подбор масштаба. Программа использует только стандартную библиотеку языка; специализированные графические библиотеки, в том числе OpenGL, не применялись. Для демонстрации интерфейса через браузер подготовлен контейнер с Xvfb и noVNC.

Проведено исследование точности и границ применимости алгоритма. Установлено, что для тел, не перекрывающихся по глубине, результат полностью совпадает с эталонным пофрагментным разрешением видимости (z-буфер): расхождение составило 0 пикселей из 630 000. При взаимном перекрытии диапазонов глубин погрешность достигает 1,41 %, а при пересечении тел – 4,58 %, причём последний случай принципиально не разрешается сортировкой целых граней и требует методов пространства изображений. Быстродействие ядра (0,038 мс для 17 граней и 0,076 мс для 34 граней) обеспечивает перерисовку сцены в реальном времени.